

Цель Блокнота

Рассмотрение механизмов интерпритации моделей машинного обучения в рамках библиотеки `scikit-learn`

- Вспоминаем про `pipeline` и обработку признаков с помощью `StandardScaler`, `OneHotEncoder` и `TargetEncoder` для `SGDRegressor`, а также Преобразование целевой переменной с помощью класса `TransformedTargetRegressor`
- Анализ весов линейной регрессии
- Для анализа ошибок модели используем `PredictionErrorDisplay`
- Бустинг `CatBoost` и анализ значимых признаков и деревьев внутри
- Для более перспективной оценки значимости признаков используем `permutation_importance`
- Для анализа вклада отдельного признака используем `PartialDependenceDisplay`
- Функция для сохранения метрик

Import библиотек

Обновим `scikit-learn` чтобы воспользоваться некими новинками

Например `TargetEncoder`

```
!pip install -U scikit-learn

Requirement already satisfied: scikit-learn in /usr/local/lib/python3.10/dist-packages (1.5.2)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (3.5.0)
```

```
!pip install catboost

Collecting catboost
  Downloading catboost-1.2.7-cp310-cp310-manylinux2014_x86_64.whl.metadata (1.2 kB)
Requirement already satisfied: graphviz in /usr/local/lib/python3.10/dist-packages (from catboost) (0.20.3)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (from catboost) (3.7.1)
Requirement already satisfied: numpy<2.0,>=1.16.0 in /usr/local/lib/python3.10/dist-packages (from catboost) (1.26.4)
Requirement already satisfied: pandas>=0.24 in /usr/local/lib/python3.10/dist-packages (from catboost) (2.2.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.10/dist-packages (from catboost) (1.13.1)
Requirement already satisfied: plotly in /usr/local/lib/python3.10/dist-packages (from catboost) (5.24.1)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from catboost) (1.16.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.24->catboost) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.24->catboost) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.24->catboost) (2024.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->catboost) (1.3.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.10/dist-packages (from matplotlib->catboost) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->catboost) (4.54.1)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->catboost) (1.4.7)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->catboost) (24.1)
Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->catboost) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->catboost) (3.1.4)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.10/dist-packages (from plotly->catboost) (9.0.0)
Downloading catboost-1.2.7-cp310-cp310-manylinux2014_x86_64.whl (98.7 MB)
98.7/98.7 MB 4.7 MB/s eta 0:00:00
Installing collected packages: catboost
Successfully installed catboost-1.2.7
```

Гигантский блок импорта

[Показать код](#)

Набор данных

Набор данных представляет собой статистику параметров автомобилей на вторичном рынке в Молдавии.

Набор включает ряд категориальных и численных значений, составляющих одну запись (строку). Число записей можно найти как число строк.

Каждый столбец в записи — это отдельный параметр.

Среди указанных параметров приведены целевой для задачи предсказания (регрессии) - цена автомобиля.

Также среди параметров есть целевой для задачи классификации - тип трансмиссии.

Последняя задача может быть рассмотрена, например, как пример задачи на заполнение пропусков (если продавец не указал соответствующий параметр).



Загрузка данных

Загружаем наши данные из файла по прямой ссылке на git-hub

```
DF = pd.read_csv('https://raw.githubusercontent.com/dayekb/Basic_ML_Alg/main/cars_moldova_clean.csv', delimiter = ',')
DF
```

	Make	Model	Year	Style	Distance	Engine_capacity(cm3)	Fuel_type	Transmission	Price(euro)
0	Toyota	Prius	2011	Hatchback	195000.0	1800.0	Hybrid	Automatic	7750.0
1	Renault	Grand Scenic	2014	Universal	135000.0	1500.0	Diesel	Manual	8550.0
2	Renault	Laguna	2012	Universal	110000.0	1500.0	Diesel	Manual	6550.0
3	Opel	Astra	2006	Universal	200000.0	1600.0	Metan/Propan	Manual	4100.0
4	Mercedes	Vito	2000	Microvan	300000.0	2200.0	Diesel	Manual	3490.0
...	...	...	...	...	...	...	...	...	...
32480	Volkswagen	Passat	2016	Sedan	88000.0	1800.0	Petrol	Automatic	11500.0
32481	Land Rover	Freelander	2002	Crossover	225000.0	1800.0	Metan/Propan	Manual	4400.0
32482	Dacia	Logan Mcv	2015	Universal	89000.0	1500.0	Diesel	Manual	7000.0
32483	Mazda	6	2006	Combi	370000.0	2000.0	Diesel	Manual	4000.0
32484	Renault	Grand Scenic	2006	Minivan	300000.0	1500.0	Diesel	Manual	4000.0

32485 rows × 9 columns

```
# Добавим пару случайных признаков для последующего анализа
DF['random_feature_1'] = np.random.random(size = DF.shape[0]) # равномерное распределение
DF['random_feature_2'] = np.random.normal(size = DF.shape[0]) # нормальное распределение
```

Список числовых и категориальных данных

```
cat_columns = ['Make', 'Model', 'Style', 'Fuel_type', 'Transmission']
num_columns = ['Year', 'Distance', 'Engine_capacity(cm3)', 'Price(euro)']
```

Предварительная обработка из [предыдущего блокнота](#)

Из предварительной обработки исключена колонка `Price(euro)`, поскольку она является целевой меткой и будет обработана отдельно

Всякие там Pipeline

Показать код

Вспомогательные функции

Показать код

Функция для сохранения результата

Показать код

```
scoring_reg = {'R2': 'r2',
               '-MSE': 'neg_mean_squared_error',
               '-MAE': 'neg_mean_absolute_error',
               '-Max': 'max_error'}

cv_rule = ShuffleSplit(n_splits=5, random_state = 42)

metrics = [mse, r2_score]

metric_names = ['mse', 'r2_score']
```

Линейная Регрессия

Считываем данные, разбиваем на тестовую и тренировочную



```
# не забываем удалить целевую переменную цену из признаков
X,y = DF.drop(columns = ['Price(euro)']), DF['Price(euro)']

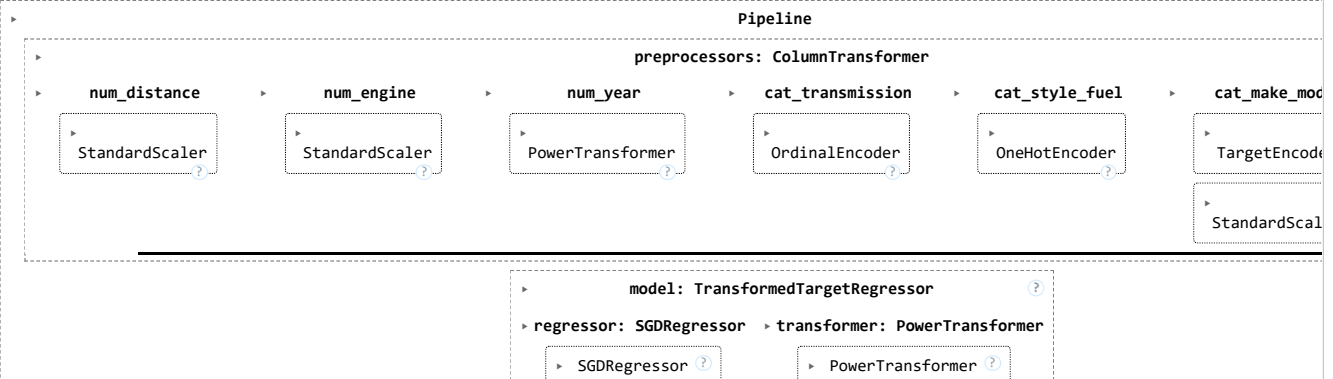
# разбиваем на тренировочную и валидационную
X_train, X_val, y_train, y_val = train_test_split(X, y,
                                                  test_size=0.3,
                                                  random_state=42)
```

Запускаем Pipeline из предварительной обработки и модели

```
pipe_all_transform = Pipeline([
    ('preprocessors', preprocessors_all),
    ('model', TransformedTargetRegressor( regressor=SGDRegressor(random_state = 42),
                                         transformer=PowerTransformer())
    )
])
```

```
%time
pipe_all_transform.fit(X_train, y_train)
```

```
CPU times: user 328 ms, sys: 10.9 ms, total: 339 ms
Wall time: 395 ms
```



Числовые признаки мы либо просто стандартизируем, либо выполняем степенное преобразование

Для категориальных признаков мы, если уникальных категорий мало, пользуемся Порядковым / OneHot кодированием; если уникальных значений много - мы пользуемся Целевым кодированием (и потом стандартизацию чтобы размер признаков был сопоставим)

Для случайных признаков мы ничего не делаем

✓ метрики модели

```
analyse_model(pipe_all_transform,
              X_train, y_train,
              X_val, y_val,
              metrics, metric_names,
              scoring_reg, cv_rule,
              model_name = 'SGDRegression',
              comments = 'num_scale+cat_TE+target_transform+random')
```



```
mse на тренировочной выборке: 14240680.4951
mse на валидационной выборке: 15671792.1704
r2_score на тренировочной выборке: 0.7658
r2_score на валидационной выборке: 0.7511
Ошибка на кросс-валидации
```

	fit_time	score_time	test_R2	test_-MSE	test_-MAE	test_-Max
0	0.381770	0.066996	0.760078	-1.420621e+07	-2163.859275	-30777.625417
1	0.384423	0.043352	0.761637	-1.541169e+07	-2222.342702	-38147.776899
2	0.480941	0.045680	0.776628	-1.371337e+07	-2133.780311	-39470.916683
3	0.427098	0.047902	0.752627	-1.477444e+07	-2243.198575	-27356.833865
4	0.403982	0.049764	0.743964	-1.499466e+07	-2194.787907	-41152.832154

```
test_R2      7.589867e-01
test_-MSE    -1.462007e+07
test_-MAE     -2.191594e+03
test_-Max     -3.538120e+04
dtype: float64
```

Метрики неплохие, но могли быть лучше

Переобучения не наблюдается, потому что train / val / cross-validation сопоставимы

✓ веса линейной модели

```
cat_style_fuel_names = pipe_all_transform['preprocessors'].transformers_[4][1]['encoder'].get_feature_names_out(cat_style_fuel)
```

```
# объединяем названия колонок в один список (важен порядок как в ColumnTransformer)
columns = np.hstack([num_distance,
                     num_engine,
                     num_year,
                     cat_transmission,
                     cat_style_fuel_names,
                     cat_make_model,
                     random,])
```

```
B0, B = get_coefs(pipe_all_transform['model'].regressor_)
features_names = list(columns)
```

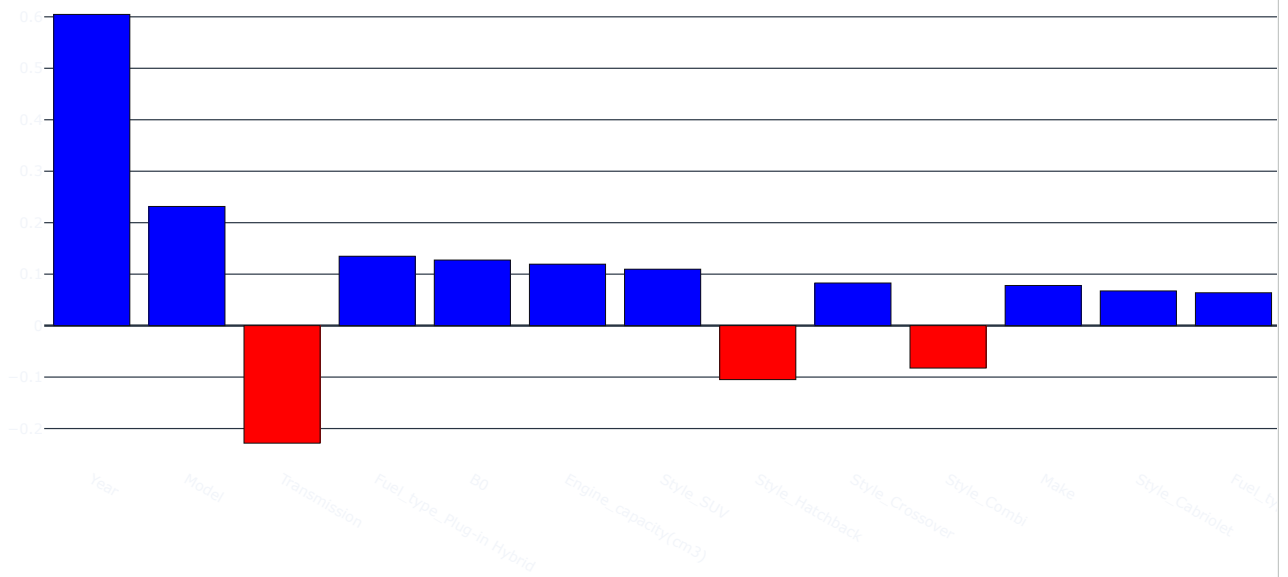
```
print_model (B0,B,features_names)
```

```
Решение
0.127-0.00*Distance+0.12*Engine_capacity(cm3)+0.60*Year-0.23*Transmission+0.07*Style_Cabriolet-0.08*Style_Combi-0.02*Style_Coupe+0.08*Style_Cros
```

```
Bs = np.hstack([B0,B])
vis_weights_threshold(Bs,columns, 0.05, width = 1400)
```



Веса модели



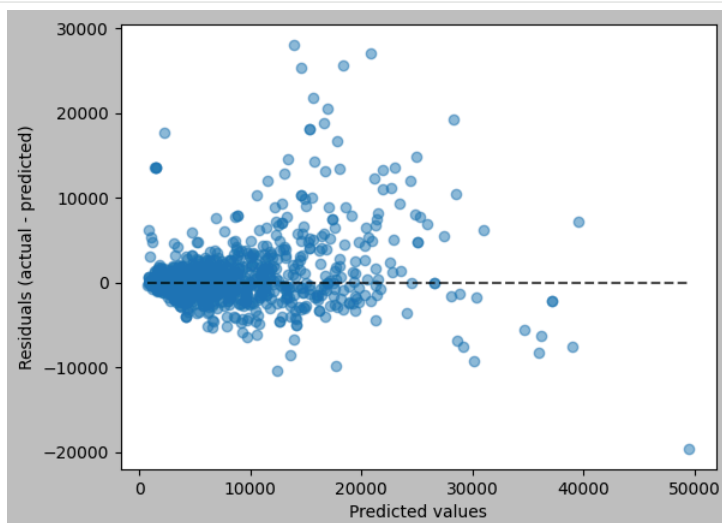
Здесь мы дополнительно отсортировали по абсолютному значению веса

Т.к. признаки одного порядка то мы "можем" сказать, что чем больше вес, тем признак значимей

В частности судя по всему хорошо работает признаки `Year`, `Model`, `Transmission`

анализ ошибок

```
PredictionErrorDisplay.from_predictions(
    y_val,
    pipe_all_transform.predict(X_val),
    kind="residual_vs_predicted",
    scatter_kwargs={"alpha": 0.5},
    line_kwargs = {"color": 'black'}
);
```



Основная масса ошибок модели - сконцентрирована в диапазоне 1-2 тысячи

Но есть и крупномасштабные ошибки

С помощью логической индексации мы можем выделить те объекты, для которых значение ошибки большое

В идеале можно детальней посмотреть на распределение признаков для этой группы



```
X_val.loc[np.abs(y_val-pipe_all_transform.predict(X_val))>10000,:]
```

	Make	Model	Year	Style	Distance	Engine_capacity(cm3)	Fuel_type	Transmission	random_feature_1	random_feature_2
3835	Porsche	Cayenne	2011	Crossover	46000.0	4800.0	Petrol	Automatic	0.284599	-1.468812
18867	Mercedes	M Class	2013	Crossover	111000.0	2143.0	Diesel	Automatic	0.380959	-0.267708
6237	Mercedes	E Class	2017	Sedan	203653.0	2000.0	Diesel	Automatic	0.537206	-1.988505
2208	Mercedes	C Class	2016	Sedan	30000.0	3000.0	Petrol	Automatic	0.350045	0.377372
2744	Lexus	GS Series	2016	Sedan	88000.0	3500.0	Hybrid	Automatic	0.678389	-1.099807
...	...	...	...	...	...	...	...	...	...	...
15963	Volkswagen	Touareg	2011	Crossover	185000.0	3000.0	Diesel	Automatic	0.565075	-0.381739
7298	Mercedes	S Class	2010	Sedan	291712.0	3000.0	Diesel	Automatic	0.502518	0.497747
13614	BMW	5 Series	2018	Sedan	29000.0	1998.0	Plug-in Hybrid	Automatic	0.062264	-1.028500
23657	Land Rover	Range Rover	2014	Crossover	190700.0	3000.0	Diesel	Automatic	0.839308	-2.150842
31896	Volvo	XC90	2017	SUV	77000.0	1997.0	Plug-in Hybrid	Automatic	0.888169	-1.428955

355 rows × 10 columns

⌵ CatBoost

Складываем в Pool

```
train_data = Pool(
    data=X_train,
    label=y_train,
    cat_features = cat_columns,
    feature_names=list(X_train.columns)
)

eval_data = Pool(
    data=X_val,
    label=y_val,
    cat_features = cat_columns,
    feature_names=list(X_train.columns)
)
```

ГОТОВИМ ОБЪЕКТ

```
model_reg = CatBoostRegressor(iterations = 1000,
    early_stopping_rounds=100,
    verbose = 100,
    cat_features = cat_columns,
    depth = 3,
    eval_metric= 'MAE'
)
```

Обучаем

```
%time
model_reg.fit(X=train_data,
    eval_set=eval_data,
    );
```

```
Learning rate set to 0.083159
0:      learn: 5341.6428495      test: 5427.6644559      best: 5427.6644559 (0) total: 138ms      remaining: 2m 17s
100:    learn: 2049.4252973      test: 2073.1290595      best: 2073.1290595 (100) total: 4.64s      remaining: 41.3s
200:    learn: 1922.6304901      test: 1953.1424278      best: 1953.1424278 (200) total: 7.26s      remaining: 28.9s
300:    learn: 1876.3369569      test: 1913.2454247      best: 1913.2454247 (300) total: 9.91s      remaining: 23s
400:    learn: 1840.3431374      test: 1884.8668050      best: 1884.8668050 (400) total: 12.5s      remaining: 18.6s
500:    learn: 1812.0139507      test: 1863.6334589      best: 1863.6334589 (500) total: 15.5s      remaining: 15.5s
600:    learn: 1789.4512229      test: 1850.3250814      best: 1850.3073400 (598) total: 18.3s      remaining: 12.1s
700:    learn: 1772.5917788      test: 1839.7862412      best: 1839.7093299 (699) total: 20.4s      remaining: 8.72s
800:    learn: 1756.1677365      test: 1828.4024918      best: 1828.3844018 (799) total: 22.9s      remaining: 5.69s
900:    learn: 1741.2288223      test: 1819.0908300      best: 1819.0908300 (900) total: 25.8s      remaining: 2.83s
999:    learn: 1727.8231370      test: 1809.0439368      best: 1809.0439368 (999) total: 28.9s      remaining: 0us
```



```
bestTest = 1809.043937
bestIteration = 999

CPU times: user 18.6 s, sys: 1.73 s, total: 20.3 s
Wall time: 29.6 s
<catboost.core.CatBoostRegressor at 0x7a841a90bbe0>
```

метрики модели

```
analyse_model(model_reg,
              X_train, y_train,
              X_val, y_val,
              metrics, metric_names,
              scoring_reg, cv_rule,
              model_name = 'CatBoost',
              comments = 'MAE_all_features+random')

mse на тренировочной выборке: 8180823.1070
mse на валидационной выборке: 9125640.9686
r2_score на тренировочной выборке: 0.8655
r2_score на валидационной выборке: 0.8550
Learning rate set to 0.065966
0: learn: 5420.9883662 total: 35ms remaining: 34.9s
100: learn: 2112.4809643 total: 2.23s remaining: 19.8s
200: learn: 1975.5569829 total: 3.51s remaining: 14s
300: learn: 1914.5623324 total: 4.25s remaining: 9.86s
400: learn: 1876.8074473 total: 5.03s remaining: 7.51s
500: learn: 1846.4224178 total: 5.8s remaining: 5.78s
600: learn: 1826.9412747 total: 6.56s remaining: 4.35s
700: learn: 1808.5331564 total: 7.33s remaining: 3.13s
800: learn: 1792.7718256 total: 8.14s remaining: 2.02s
900: learn: 1780.1507718 total: 8.96s remaining: 985ms
999: learn: 1768.4875738 total: 9.74s remaining: 0us
Learning rate set to 0.065966
0: learn: 5391.5222248 total: 10.4ms remaining: 10.4s
100: learn: 2098.3675546 total: 810ms remaining: 7.21s
200: learn: 1954.5253552 total: 2.4s remaining: 9.55s
300: learn: 1890.4790204 total: 4.01s remaining: 9.32s
400: learn: 1852.5784395 total: 4.91s remaining: 7.34s
500: learn: 1827.1931147 total: 5.7s remaining: 5.67s
600: learn: 1807.6835599 total: 6.49s remaining: 4.31s
700: learn: 1792.2717298 total: 7.34s remaining: 3.13s
800: learn: 1775.4542035 total: 8.16s remaining: 2.03s
900: learn: 1760.9060928 total: 9s remaining: 989ms
999: learn: 1747.5554644 total: 9.82s remaining: 0us
Learning rate set to 0.065966
0: learn: 5402.2197817 total: 10.8ms remaining: 10.8s
100: learn: 2102.8355024 total: 818ms remaining: 7.28s
200: learn: 1999.4248957 total: 1.53s remaining: 6.07s
300: learn: 1943.7325529 total: 2.2s remaining: 5.1s
400: learn: 1900.4962523 total: 2.92s remaining: 4.35s
500: learn: 1865.7562606 total: 3.69s remaining: 3.67s
600: learn: 1835.6132997 total: 4.83s remaining: 3.21s
700: learn: 1814.7486328 total: 6.63s remaining: 2.83s
800: learn: 1792.2218886 total: 7.97s remaining: 1.98s
900: learn: 1776.6350796 total: 8.83s remaining: 970ms
999: learn: 1758.4035724 total: 9.63s remaining: 0us
Learning rate set to 0.065966
0: learn: 5420.2848145 total: 10.1ms remaining: 10.1s
100: learn: 2068.5714443 total: 815ms remaining: 7.25s
200: learn: 1944.9227031 total: 1.58s remaining: 6.29s
300: learn: 1882.0335913 total: 2.34s remaining: 5.43s
400: learn: 1847.0695856 total: 3.14s remaining: 4.7s
500: learn: 1818.8399974 total: 3.97s remaining: 3.95s
600: learn: 1789.3964118 total: 4.76s remaining: 3.16s
700: learn: 1770.4445332 total: 5.57s remaining: 2.38s
800: learn: 1753.0064851 total: 6.38s remaining: 1.58s
900: learn: 1739.9282590 total: 7.22s remaining: 793ms
999: learn: 1728.9069976 total: 8.56s remaining: 0us
Learning rate set to 0.065966
0: learn: 5427.8520889 total: 21.3ms remaining: 21.2s
100: learn: 2075.1591079 total: 1.82s remaining: 16.2s
200: learn: 1950.6163086 total: 2.87s remaining: 11.4s
300: learn: 1893.4899576 total: 3.63s remaining: 8.43s
400: learn: 1858.6329890 total: 4.38s remaining: 6.55s
500: learn: 1829.9175512 total: 5.21s remaining: 5.19s
600: learn: 1806.9996279 total: 6.06s remaining: 4.02s
700: learn: 1784.3723902 total: 6.89s remaining: 2.94s
800: learn: 1767.8693674 total: 7.74s remaining: 1.92s
900: learn: 1757.0960991 total: 8.58s remaining: 943ms
999: learn: 1743.5507405 total: 9.41s remaining: 0us
```

Метрики получили

Переводим в таблицу

Feature Importance

```
fit_time score_time test_R2 test_-MSE test_-MAE test_-Max

def feature_importance_plotter(model, features_names):
    # У CatBoost значимость признаков не нормированная
    feature_importance = model.feature_importances_/np.sum(model.feature_importances_)

    sorted = np.argsort(feature_importance)
```

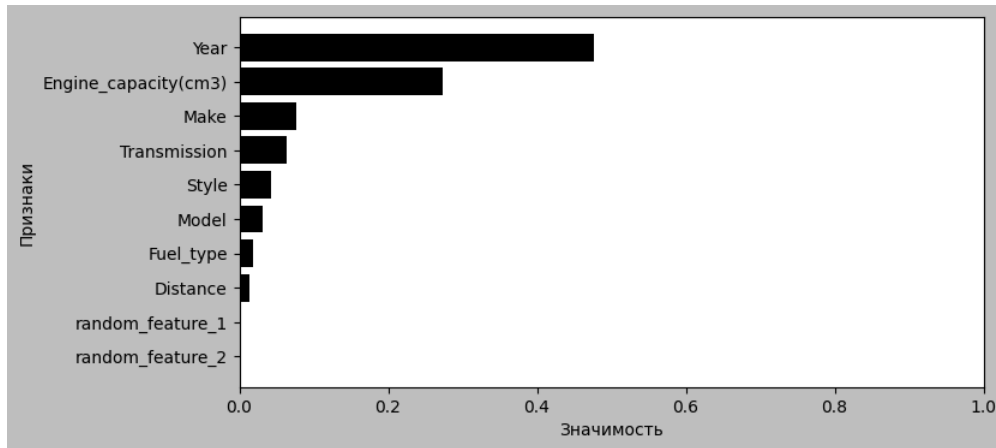
```

ypos = np.arange(len(features_names))

fig= plt.figure(figsize=(8,4))
plt.barh(ypos, feature_importance[sorted])
plt.xlim([0,1])
plt.ylabel('Признаки')
plt.xlabel('Значимость')
plt.yticks(ypos,features_names[sorted] );

```

```
feature_importance_plotter(model_reg, np.array(X_train.columns))
```



А вот тут важнее `Year`, `Engine_capacity(cm3)`, признаки `Make` и `Transmission` чуть менее важны

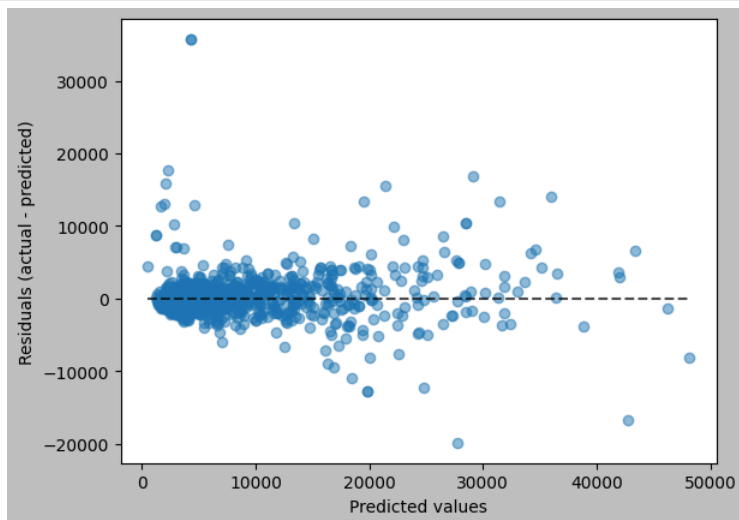
Радует что случайные признаки в нуле

▼ анализ ошибок

```

PredictionErrorDisplay.from_predictions(
    y_val,
    model_reg.predict(eval_data),
    kind="residual_vs_predicted",
    scatter_kwargs={"alpha": 0.5},
    line_kwargs = {"color": 'black'}
);

```



тут в среднем получше

Основные проблемы с дорогими автомобилями, которых не так много

```
X_val.loc[np.abs(y_val-model_reg.predict(eval_data))>10000,:]
```




	Make	Model	Year	Style	Distance	Engine_capacity(cm3)	Fuel_type	Transmission	random_feature_1	random_feature_2
2001	BMW	3 Series	2018	Sedan	45000.0	2000.0	Plug-in Hybrid	Automatic	0.558813	-0.318845
25194	Audi	A6	2017	Sedan	222222.0	2700.0	Diesel	Manual	0.571273	-0.516804
25330	Nissan	Sunny	1995	Sedan	37998.0	1392.0	Metan/Propan	Manual	0.777424	-0.182151
27147	KIA	Carnival	2018	Minivan	67680.0	2199.0	Diesel	Automatic	0.544479	0.222315
19542	Volvo	XC60	2016	SUV	40000.0	2400.0	Diesel	Automatic	0.083988	-2.649413
...	...	...	...	...	...	...	...	...	...	...
6127	Mercedes	S Class	2014	Sedan	106029.0	2987.0	Diesel	Automatic	0.219918	0.361065
29327	Hyundai	Tucson	2019	SUV	25000.0	2000.0	Diesel	Automatic	0.699209	0.717066
21433	BMW	7 Series	2014	Sedan	74000.0	4400.0	Petrol	Automatic	0.397215	1.117336
23657	Land Rover	Range Rover	2014	Crossover	190700.0	3000.0	Diesel	Automatic	0.839308	-2.150842
31896	Volvo	XC90	2017	SUV	77000.0	1997.0	Plug-in Hybrid	Automatic	0.888169	-1.428955

175 rows × 10 columns

Глубина дерева для Отрисовки

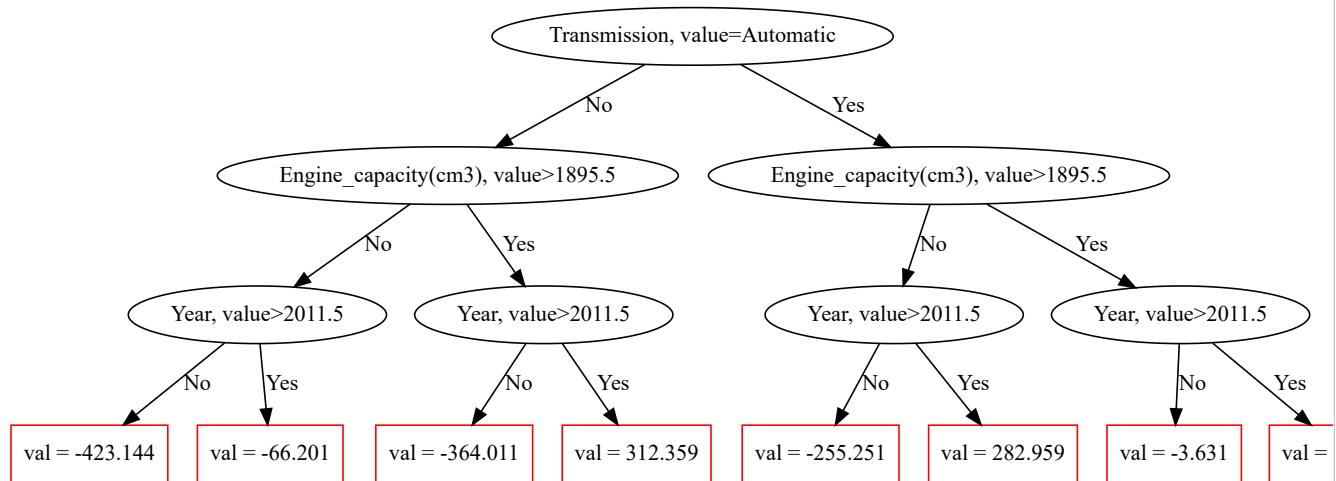
```
##title Глубина дерева для Отрисовки{ run: "auto" }
tree_idx=0 ##param {type:"slider", min:0, max:25, step:1}

model_reg.plot_tree(tree_idx=tree_idx,
pool = train_data)
```

tree_idx

0

Скрыть код



Permutation importance

Есть в `scikit-learn`

Это про то что будет если какую-то переменную перемешать (в рамках столбца). Если от перемешивания результат меняется - то наверное признак важный

```
from sklearn.inspection import permutation_importance
```

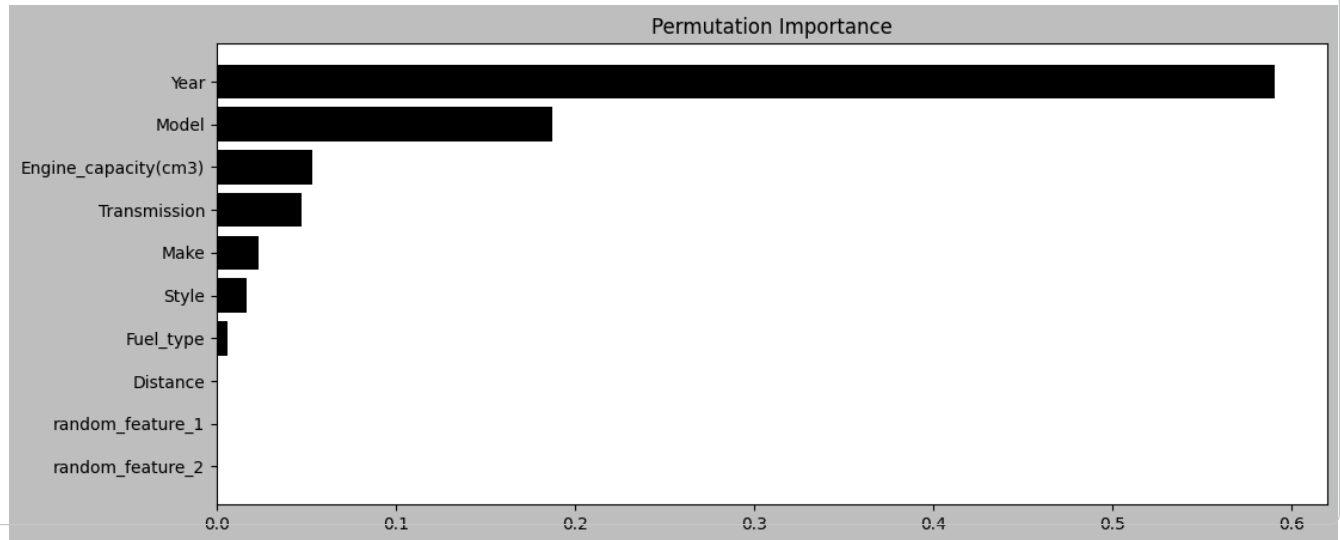
```
%time
perm_importance_lin = permutation_importance(pipe_all_transform, # модель / pipeline
X_val, # признаки
y_val, # метки
n_repeats=5, # сколько раз делать перемешивание
random_state=42)
```

```
CPU times: user 3.4 s, sys: 2.29 s, total: 5.69 s
Wall time: 5.96 s
```

Визуализируем как `feature importance` от деревьев решений / бустингов

```
def permutation_importance_plotter(perm_importance, cols):
    sorted_idx = perm_importance.importances_mean.argsort()
    fig = plt.figure(figsize=(12, 5))
    plt.barh(range(len(sorted_idx)), perm_importance.importances_mean[sorted_idx], align='center')
    plt.yticks(range(len(sorted_idx)), cols[sorted_idx])
    plt.title('Permutation Importance');
```

```
permutation_importance_plotter(perm_importance_lin, np.array(X_val.columns))
```



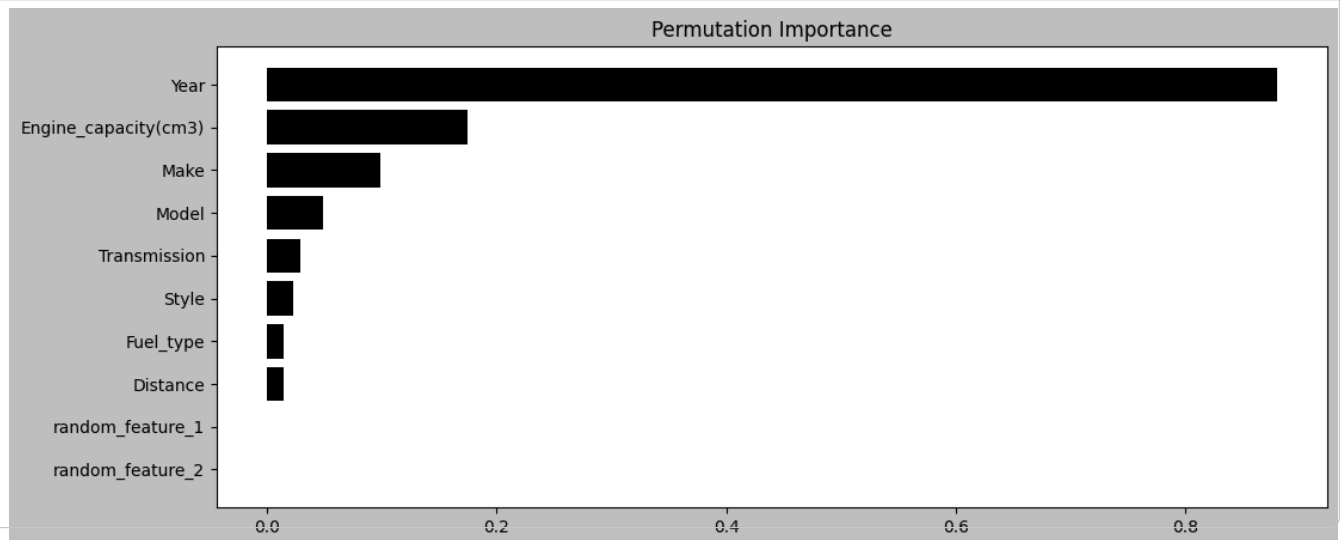
для линейной модели первые два признака оказались значимыми как и в весах модели

При этом поднялся `Engine_capacity(cm3)`

```
%time
perm_importance_cb = permutation_importance(model_reg, # модель / pipeline
                                           X_val, # признаки
                                           y_val, # метки
                                           n_repeats=5, # сколько раз делать перемешивание
                                           random_state=42)
```

```
CPU times: user 3.07 s, sys: 80.2 ms, total: 3.15 s
Wall time: 2.7 s
```

```
permutation_importance_plotter(perm_importance_cb, np.array(X_val.columns))
```



Для бустинга история вроде совпадает с простым Feature Importance

Partial plots

История о том как отдельный признак влияет на целевую метку

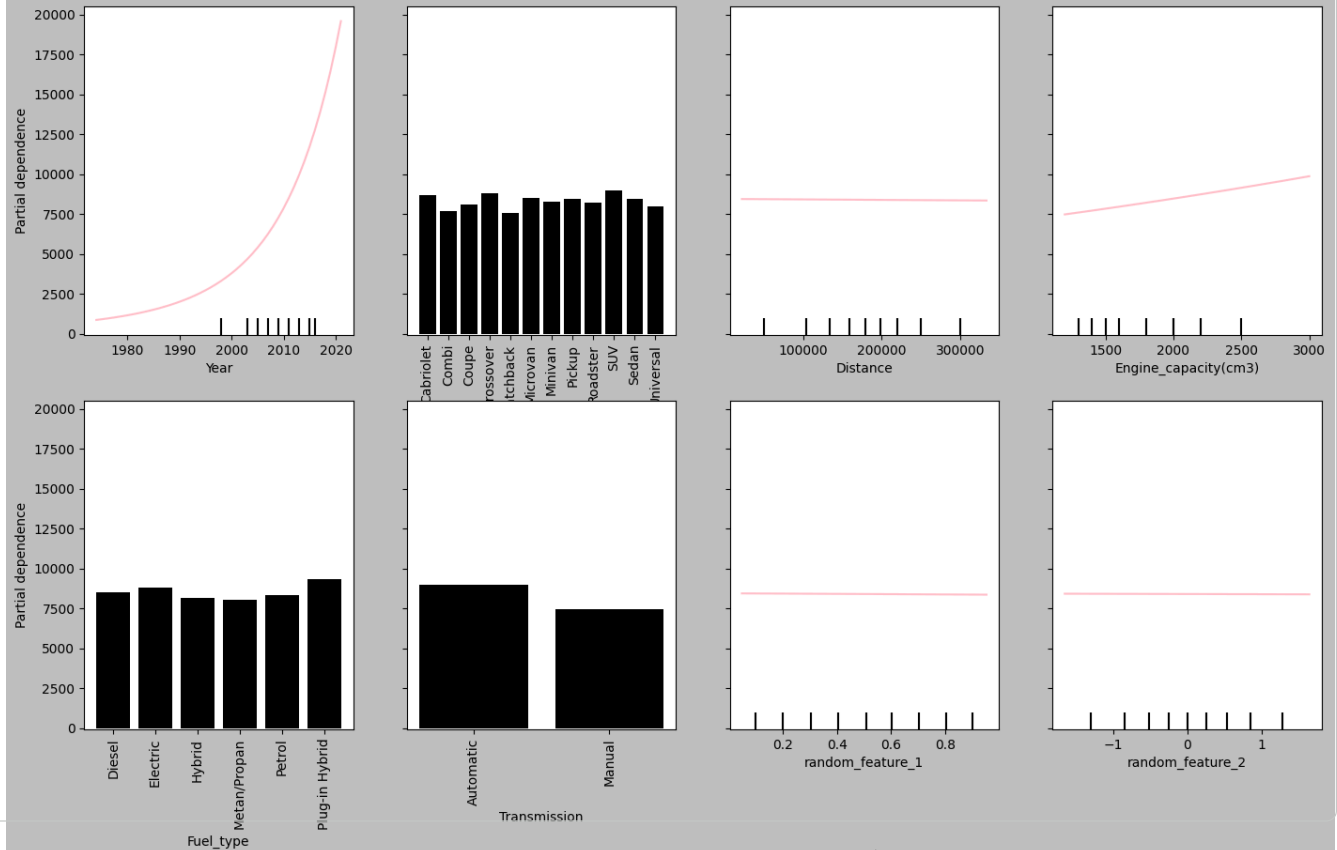
```
from sklearn.inspection import PartialDependenceDisplay
```

```

%%time
fig, ax = plt.subplots(figsize=(17, 10))
PDD = PartialDependenceDisplay.from_estimator(pipe_all_transform, # модель / pipeline
                                             X_val, # признаки
                                             features = X_val.columns[2:], # какие признаки проверять будем
                                             categorical_features = cat_columns, # какие признаки категориальные
                                             kind="average",
                                             n_cols = 4, # для отрисовки - сколько столбцов
                                             random_state=42,
                                             line_kw = {"color" : 'pink'},
                                             ax=ax);

```

CPU times: user 28.4 s, sys: 18.8 s, total: 47.2 s
Wall time: 38.1 s



Для линейной модели наблюдаем подобие линейной зависимости для объема двигателя и пробега (хотя так себе, если честно для пробега) и близкое к экспоненциальному зависимость от года (не зря делали степенное преобразование)

Для разных категорий наблюдаются разные цены, но это мы и раньше могли догадываться

```

%%time
fig, ax = plt.subplots(figsize=(17, 10))
PDD = PartialDependenceDisplay.from_estimator(model_reg, # модель / pipeline
                                             X_val, # признаки
                                             features = X_val.columns[2:], # какие признаки проверять будем
                                             categorical_features = cat_columns, # какие признаки категориальные
                                             kind="average",
                                             n_cols = 4, # для отрисовки - сколько столбцов
                                             random_state=42,
                                             ax=ax);

```



CPU times: user 26.9 s, sys: 1.01 s, total: 27.9 s
Wall time: 18.8 s

